

零、CVAT 环境配置(首次使用)

1. 安装 Docker Desktop

CVAT 使用 Docker 运行。

需要先安装：

Docker Desktop

安装完成后，确认：

`docker`

`docker compose`

命令可以在终端正常运行。

2. 下载 CVAT

推荐直接 clone 官方 CVAT：

`git clone https://github.com/cvat-ai/cvat.git`

例如放在：

`C:\Users\<username>\cvat`

3. 首次启动 CVAT

进入 CVAT 根目录：

`cd "C:\Users\<username>\cvat"`

启动：

`docker compose up -d`

首次启动会：

自动下载 Docker 镜像

初始化数据库

创建 CVAT 服务

第一次启动时间可能较长。

4. 创建管理员账号

首次启动后，需要创建管理员账号：

```
docker exec -it cvat_server bash -ic 'python3 ~/manage.py createsuperuser'
```

按照提示输入：

username

email

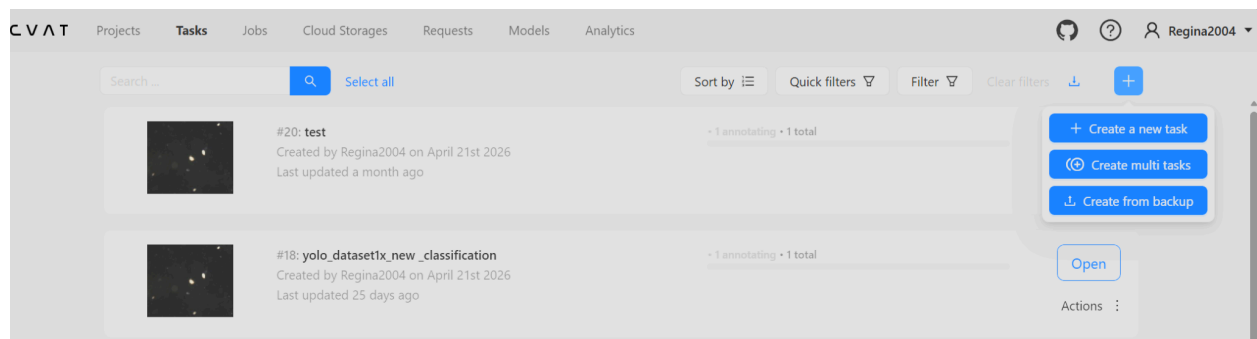
password

5. 进入 CVAT 页面

浏览器打开：

<http://localhost:8080>

使用刚刚创建的账号登录。



在task里右上角点击+按钮，选择create a new task

Create a new task

Basic configuration

* Name

YOLO_dataset ✓

Project

Select project

Labels

Raw
Constructor

daxingzao ✓

Rectan...

●

Add an attribute ⊕

Continue

Cancel

* Select files

My computer
Connected file share
Remote sources
Cloud Storage

Click or drag files to this area

You can upload an archive with images, a video, or multiple images

给task取名，在labels那块**按顺序**添加类别名，**选择rectangle**，颜色随意自选。然后**把图片打包成zip**拖入下方区域。（截图数据集构建参考下方教程）

目前的类别顺序，如果有新类别则往后添加。（YOLO训练的时候每个类别里都要有内容。不能data.yaml里有那个类别，但是数据集里没有，所以在创建数据集时要想好是否包含曾经的数据集，如果包含可直接往后添加类别，如果不包含，则要考虑重新设定编号与类别的映射）

- 0: daxingzao
- 1: jianshuizao
- 2: xiannvchong
- 3: lunchong
- 4: xiangbizao
- 5: weizhi
- 6: xianchong

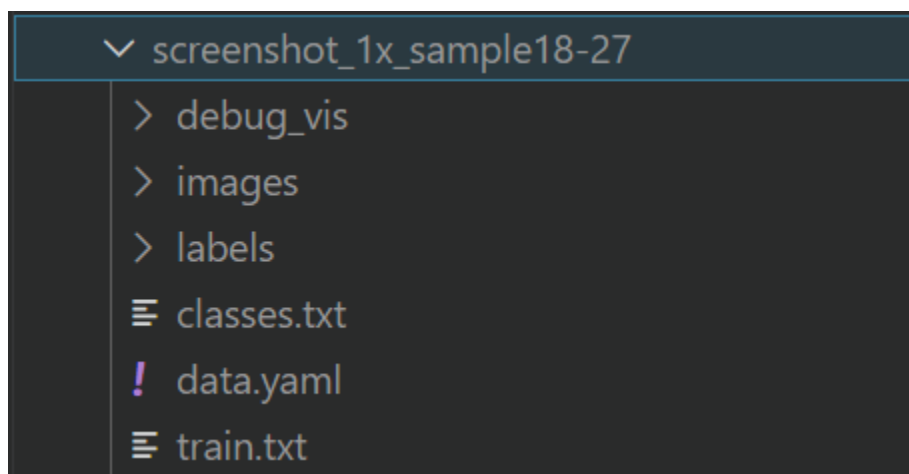
自动获取截图数据集的方法（截图+模型自动画框做初步为标签）

```

"""
Jetson中运行方式：
cd ~/plankton-detection
source venv_jetson/bin/activate
PYTHONPATH=. python3 scripts/dataset/get_screenshot_dataset.py
"""

```

获取初始为标签截图



运行后output格式如上, debug_vis用于检查标签效果, 不能上传到cvat。

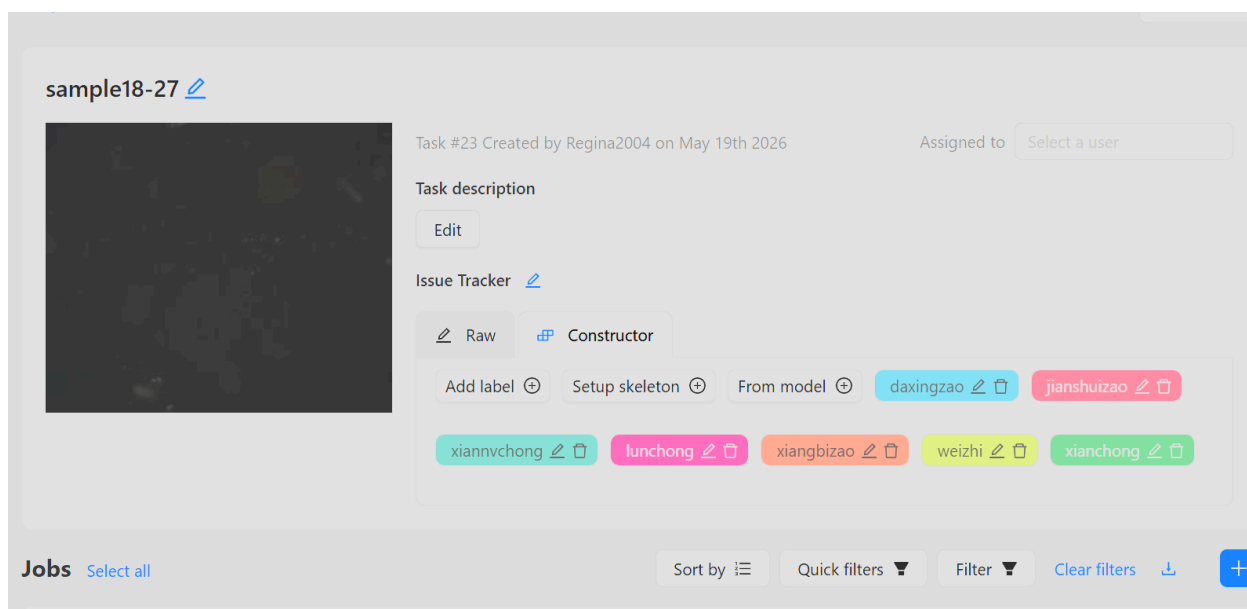
名称	修改日期	类型	大小
images	2026/5/19 11:03	文件夹	
labels	2026/5/19 11:03	文件夹	
data	2026/5/19 11:03	Yaml 源文件	1 KB
train	2026/5/19 11:03	文本文档	8 KB

下载到本地, 删除debug_vis和classes.txt, 压缩成.zip文件夹。

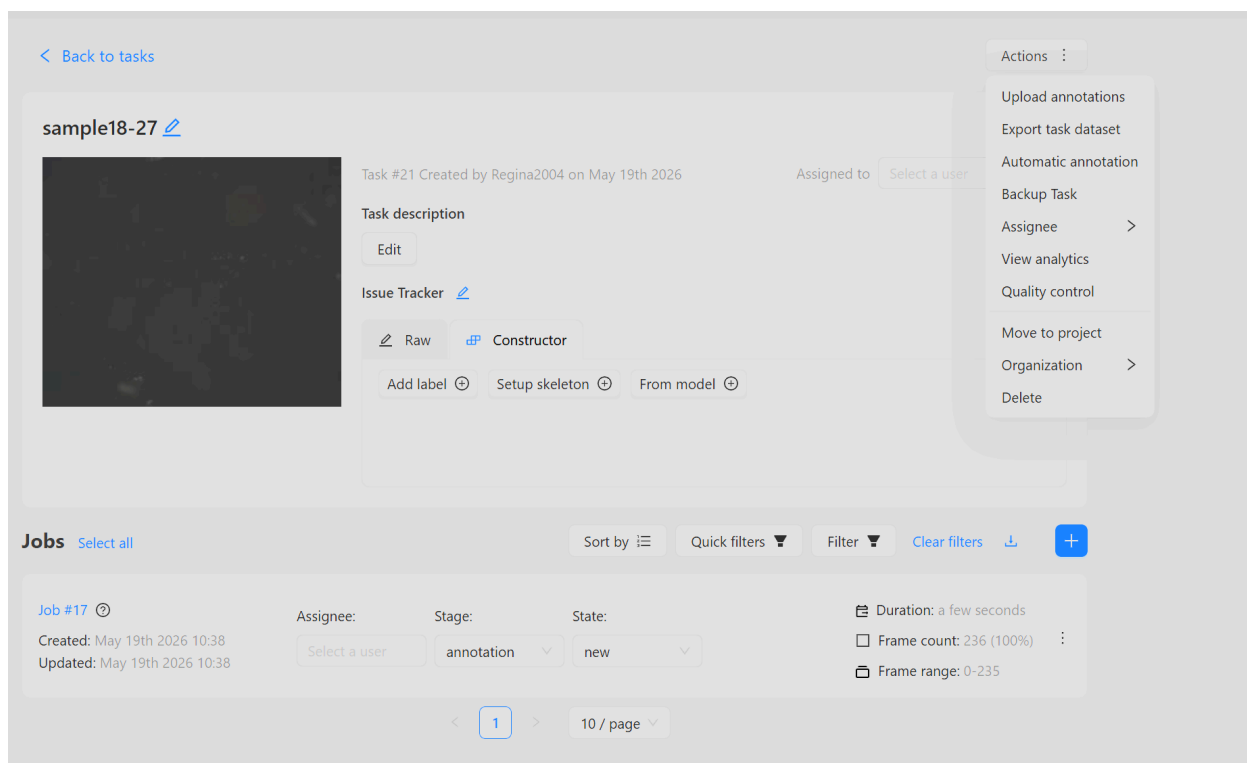
目录格式参考:

zip 根目录

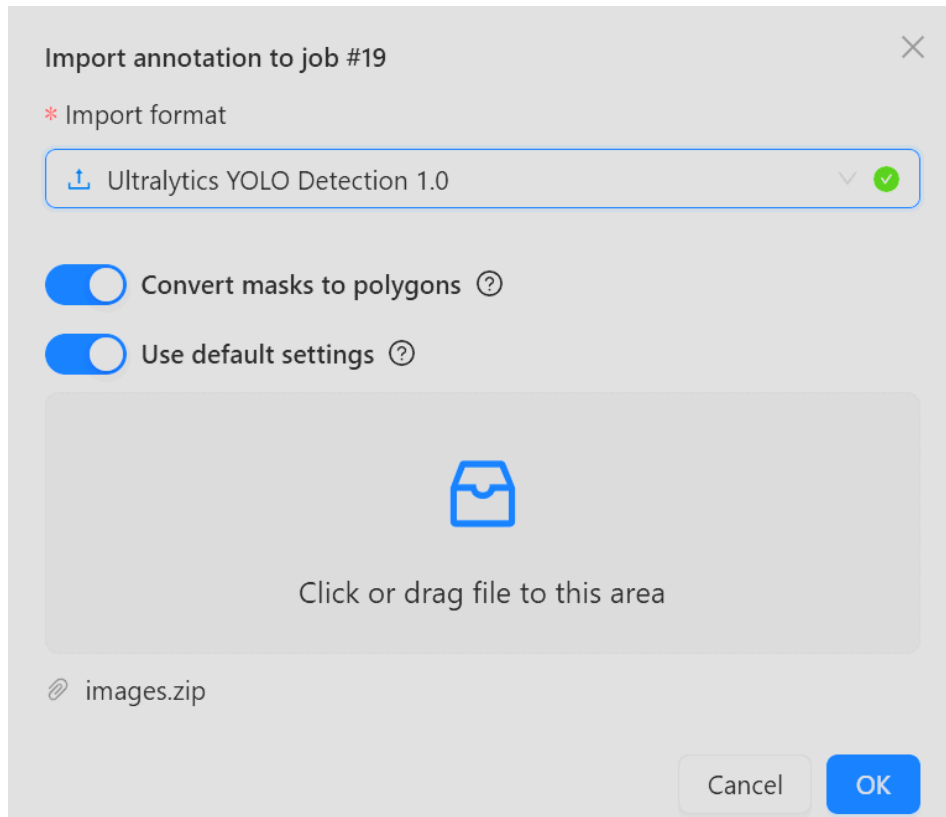
```
├── data.yaml
├── train.txt
├── images/
│   └── train/
│       └── sample20_000020.png
├── labels/
│   └── train/
│       └── sample20_000020.txt
```



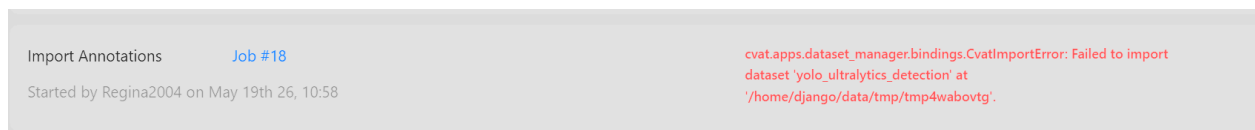
这样就有了images和初始设置的labels



还需要把之前用模型识别的labels数据挂到上面，再专门上传一遍annotations
点击右侧Actions → upload annotations



选择**Ultralytics YOLO Detection 1.0**,把刚刚压缩过的**.zip**再次上传。就可以自动匹配所有的截图和上面的标注了。接下来可以人工修改所有的label。



如果有这种报错，可能是data.yaml的格式不对，查看是否符合以下格式：

```
path: ./
train: train.txt
names:
  0: daxingzao
  1: jianshuizao
  2: xiannvchong
  3: lunchong
  4: xiangbizao
  5: weizhi
  6: xianchong
```

一、CVAT 数据标注后导出

进入 CVAT 项目目录并启动 Docker:

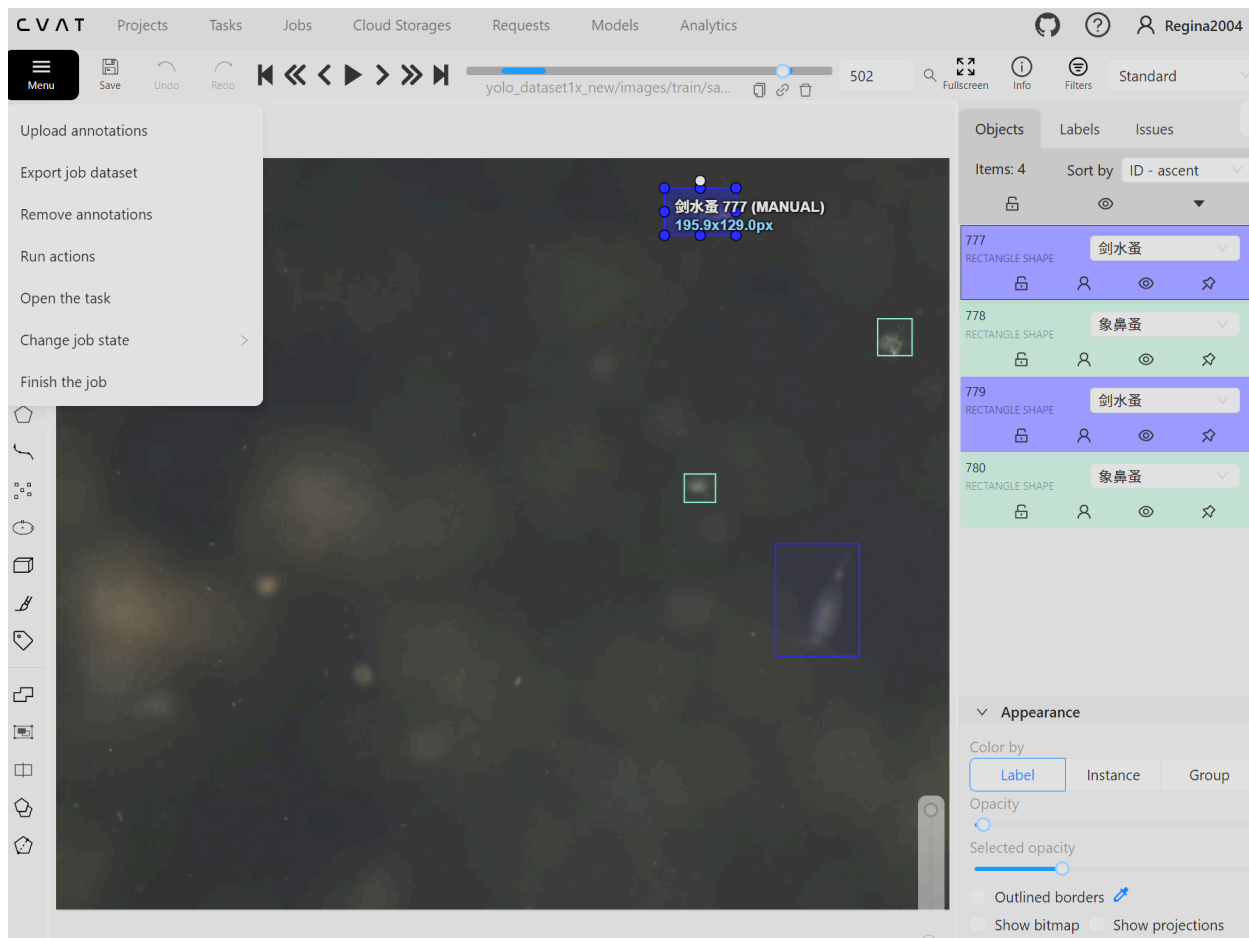
```
cd "C:\Users\<username>\cvat"
```

```
docker compose up -d
```

```
(base) PS C:\Windows\system32> cd "C:\Users\Regina Sun\cvat"
(base) PS C:\Users\Regina Sun\cvat> docker compose up -d
[+] up 18/18
✓ Container cvat_redis_ondisk      Running      0.0s
✓ Container cvat_opa              Running      0.0s
✓ Container cvat_grafana          Running      0.0s
✓ Container cvat_redis_inmem      Running      0.0s
✓ Container cvat_clickhouse       Running      0.0s
✓ Container traefik               Running      0.0s
✓ Container cvat_vector           Running      0.0s
✓ Container cvat_db              Running      0.0s
✓ Container cvat_worker_export    Running      0.0s
✓ Container cvat_worker_utils     Running      0.0s
✓ Container cvat_worker_consensus Running      0.0s
✓ Container cvat_worker_import    Running      0.0s
✓ Container cvat_worker_annotation Running      0.0s
✓ Container cvat_worker_chunks    Running      0.0s
✓ Container cvat_server           Running      0.0s
✓ Container cvat_worker_quality_reports Running      0.0s
✓ Container cvat_worker_webhooks   Running      0.0s
✓ Container cvat_ui               Running      0.0s
```

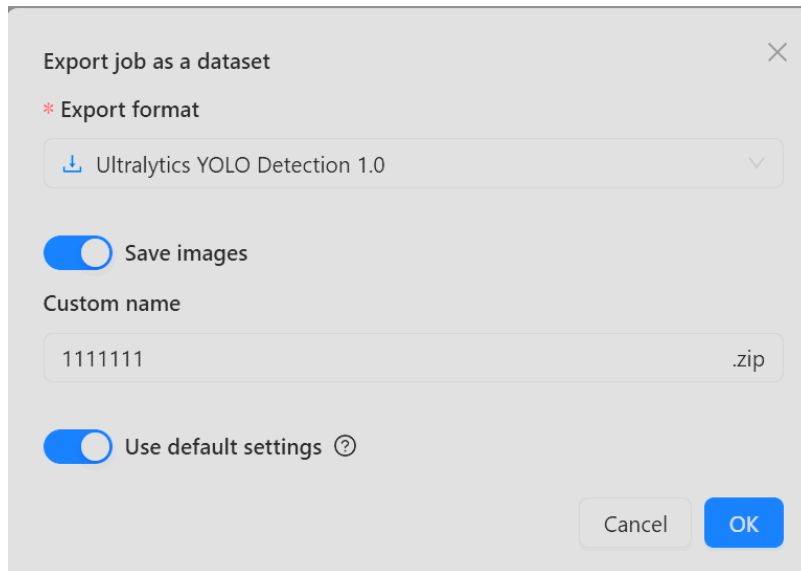
启动完成后，在浏览器进入 CVAT 页面。

在标注界面点击：



Menu → Export job dataset

推荐导出格式：



Ultralytics YOLO Detection 1.0

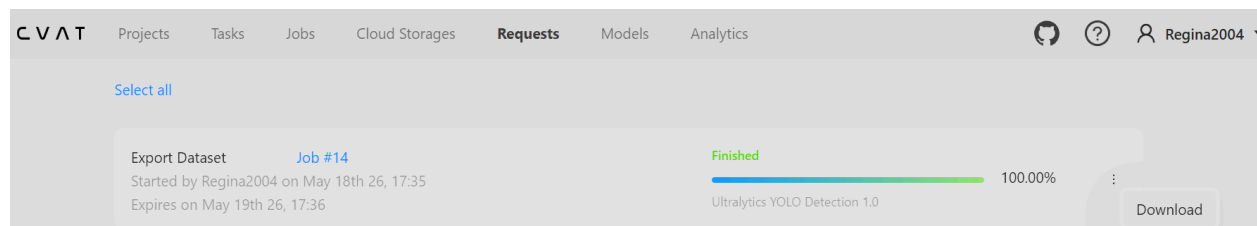
其中：

Save images:

- 开启：同时导出图片与 annotation
- 关闭：仅导出 annotation，导出速度更快

如果关闭 Save images，则需要后续手动整理图片目录，并保证图片与 annotation 正确对应。

导出任务创建后，可以在：



Requests

页面查看导出进度，并下载最终生成的数据集 .zip 文件。点击右侧三个点下载文件。

二、导出以后需要重构干净目录

CVAT 导出的 YOLO 数据集可能存在以下问题(案例)：


```
▼ 1111111
  > images\train\yolo_dataset1x_new\images\train
  > labels\train\yolo_dataset1x_new\images\train
  ! data.yaml
  ≡ train.txt
```

- 多层嵌套目录
- train.txt / val.txt 混杂
- 图片与 label 路径重复嵌套
- 导出目录结构不统一
- 不方便后续训练与部署

为了保证：

- YOLO 训练稳定
- Windows / Jetson 兼容
- 后续推理部署统一
- 实验室多人协作维护方便

需要将原始导出数据重新整理为标准 YOLO 数据集结构。

三、标准 YOLO 数据集结构

整理后的标准数据集结构如下：

```
▼ yolo_dataset1x_merged_classification
  ▼ images
    > train
    > val
  ▼ labels
    > train
    > val
  ! data.yaml
```

```
data/<dataset_name>/
├── images/
│   ├── train/
│   └── val/
├── labels/
│   ├── train/
│   └── val/
├── data.yaml
└── dataset_report.txt
```

其中：

images/train
images/val

存放图片。

labels/train
labels/val

存放 YOLO detection label。

所有图片与 label 必须同名。

例如：

sample15_000005.png
sample15_000005.txt

必须一一对应。

四、data.yaml 类别配置

```

data > 1111111 > ! data.yaml
1  names:
2    0: "\u5927\u578B\u6E9E"
3    1: "\u5251\u6C34\u86A4"
4    2: "\u4ED9\u5973\u866B"
5    3: "\u8F6E\u866B"
6    4: "\u8C61\u9F3B\u86A4"
7    5: "\u672A\u77E5"
8    6: "\u6C34\u8785"
9    7: "\u8349\u5C65\u866B"
10   8: "\u7EBF\u866B"
11  path: .
12  train: train.txt
13  |

```

如图，如果CVAT 导出的中文类别名称无法被正确识别，则需手动重写 `data.yaml`。

示例：

```

path: .
train: images/train
val: images/val

```

nc: 7

```

names:
0: daxingzao
1: jianshuizao
2: xiannvchong
3: lunchong
4: xiangbizao
5: weizhi
6: xianchong

```

说明：

nc:
类别数量

names:
类别 id 与类别名称的映射关系

如果后续新增新的微生物类别，可以继续向后添加新的 class id。

五、重构干净目录

1. 初始输入目录

原始数据放在：

data/<dataset_name>/

目录结构允许比较混乱，例如：

```
data/<dataset_name>/
├── images/train/images/trainsample15_000005.png
├── labels/train/labels/train/sample15_000005.txt
├── data.yaml
└── train.txt
```

只需要满足以下条件：

1. 图片与 txt 同名
 2. data.yaml 已经人工写好类别
-

2. 数据集重构脚本

脚本位置：

scripts/dataset/build_clean_yolo_dataset.py

运行方式：

```
PYTHONPATH=. python3 scripts/dataset/build_clean_yolo_dataset.py
```

3. 脚本功能

该脚本会：

递归扫描 raw_cvat_export

↓

找到所有图片

↓

找到所有 label txt

↓
过滤 train.txt / val.txt / dataset_report.txt 等非 label 文件
↓
按文件名 stem 匹配图片与 txt
↓
检查 label 格式
↓
检查 class id 是否存在于 data.yaml
↓
检查 bbox 是否为 0~1 归一化坐标
↓
保留空 txt, 并记录为空图
↓
随机划分 train / val
↓
复制到新的标准 YOLO 数据集目录
↓
生成新的 data.yaml
↓
生成 dataset_report.txt

脚本不会修改原始数据。

4. 输出目录

脚本最终输出：

data/<dataset_name>/

例如：

data/yolo_dataset1x_classification_v2/

结构为：

yolo_dataset1x_classification_v2/

```
|— images/  
|   |— train/  
|   |— val/  
|— labels/  
|   |— train/  
|   |— val/  
|— data.yaml
```

5. dataset_report.txt

该文件仅用于：

数据检查
数据统计
问题排查

不会影响 YOLO 训练。

主要记录：

图片数量
label 数量
成功匹配数量
缺失 label
孤立 label
空图数量
异常 label 行
类别 bbox 数量统计
train / val 数量

六、检查数据集(可选)

在正式训练前，建议运行数据集检查脚本。

该脚本不会修改数据集，仅用于最终检查。

脚本位置：

scripts/dataset/check_yolo_dataset.py

运行方式：

```
PYTHONPATH=. python3 scripts/dataset/check_yolo_dataset.py
```

1. 输入要求

输入必须是已经整理好的标准 YOLO 数据集：

```
data/<dataset_name>/
├── images/
│   ├── train/
│   └── val/
├── labels/
│   ├── train/
│   └── val/
└── data.yaml
```

2. 检查内容

脚本会检查：

1. data.yaml 是否存在
 2. names / nc 是否正确
 3. images/train 是否都有对应 label
 4. images/val 是否都有对应 label
 5. 是否存在孤立 label
 6. train / val 是否有重复样本
 7. label 每行是否为 5 列
 8. class id 是否存在于 data.yaml
 9. bbox 是否为 0~1 归一化坐标
 10. 空 txt 数量
 11. 每个类别的 bbox 数量
-

3. 终端输出

检查结果会直接打印在终端。

正常情况示例：

```
===== YOLO Dataset Check =====
```

```
[PASS] data.yaml loaded
```

```
[PASS] train image-label pairs
```

```
[PASS] val image-label pairs
```

```
[PASS] train/val split
```

[WARNING] empty label files

[PASS] label format

===== CHECK PASSED =====

Dataset is ready for YOLO training.

如果存在严重错误, 例如:

- 缺失 label
- class id 越界
- bbox 非归一化
- train / val 数据泄漏

脚本会打印:

[ERROR] ...

并终止运行。

七、YOLO 模型训练

1. 训练前要求

训练前需要已经完成:

1. CVAT 标注
2. build_clean_yolo_dataset.py
3. check_yolo_dataset.py(可选)

2. 训练脚本

脚本位置:

scripts/train/train_yolo.py

运行方式:

```
PYTHONPATH=. python scripts/train/train_yolo.py
```

3. 主要配置

训练脚本中的主要配置：

DATASET_NAME

RUN_NAME

MODEL_NAME

EPOCHS

IMGSZ

BATCH

WORKERS

DEVICE

PATIENCE

说明：

DATASET_NAME:

对应 data/ 下的数据集目录名称

RUN_NAME:

对应 runs/ 下的训练输出目录

MODEL_NAME:

预训练模型，例如 yolov8n.pt

4. 训练输出

训练结果默认输出到：

runs/<RUN_NAME>/

其中：

weights/best.pt

为验证集表现最好的模型。

后续推理与 Jetson 部署均使用该模型。

八、整体 Pipeline

完整流程如下：

CVAT 标注

↓

导出 Ultralytics YOLO Detection 数据集

↓

build_clean_yolo_dataset.py

↓

获得了标准 YOLO 数据集

↓

check_yolo_dataset.py(可选)

↓

train_yolo.py

↓

best.pt(可后续转化为engine, 代码再scripts里)

↓

main.py / (开始追踪识别)